WebRTC Masterclass — Appendix

Reference Material

Robert Hess

SharkFest'26 Europe

Contents

- Glossary 4
- ICE candidates 6
- Display & Capture Filters 7
  - Useful display filters 7
  - Capture filters 7
- My Capture setup 9
  - Setup at a glance 9
  - Most relevant configurations 10
- MS Teams Cheat Sheet 11
  - Connect String 11
  - Geo Distribution 11
  - Signaling 11
  - Log files 11
  - Ports & Addresses 11
  - Domains 12
  - Display Filters 12
  - ICE Grid 13
- MS Teams Web Call Setup at a Glance 14
  - Setup sequence 14
  - Servers and selection 14
  - ICE: trickle, lite and NAT 15
  - Codecs 15
  - Different from mainline WebRTC 16
- Zoom 17
  - Cheat Sheet 17
    - Connect String 17
    - Geo Distribution 17
    - Signaling 17
    - Log files 17
    - Ports & Addresses 17
    - Domains (TCP 443) 18
    - Display Filters 18

ICE Grid 18
Zoom Web Call Setup at a Glance 19
Setup sequence 19
Servers and selection 19
ICE: trickle, lite and NAT 20
Codecs 20
Different from mainline WebRTC 21
Webex 22
Cheat Sheet 22
Connect String 22
Geo Distribution 22
Signaling 22
Log files 22
Ports & Addresses 22
Domains (TCP 443) 23
Display Filters 23
ICE Grid 24
Webex Web Call Setup at a Glance 25
Setup sequence 25
Servers and selection 26
ICE: trickle, lite and NAT 26
Codecs 26
Different from mainline WebRTC 27
Google Meet 28
Cheat Sheet 28
Connect String 28
Geo Distribution 28
Signaling 28
Log files 28
Ports & Addresses 28
Domains (TCP 443) 29
Display Filters 29
ICE Grid 30
Google Meet Web Call Setup at a Glance 31
Setup sequence 31
Servers and selection 31
ICE: trickle, lite and NAT 32
Codecs 32
Different from mainline WebRTC 32
GoToMeeting 34
Cheat Sheet 34
Connect String 34
Geo Distribution 34
Signaling 34
Log files 34

Ports & Addresses 34
Domains (TCP 443) 35
Display Filters 35
ICE Grid 36
GoToMeeting Web Call Setup at a Glance 37
Setup sequence 37
Servers and selection 37
ICE: trickle, lite and NAT 38
Codecs 38
Different from mainline WebRTC 39
References 40
Class related: 40
Articles, RFCs 40
SharkFest talks 40
Tooling 40
WebRTC-internals / rtcstats 40
TURN server security 40
People 41

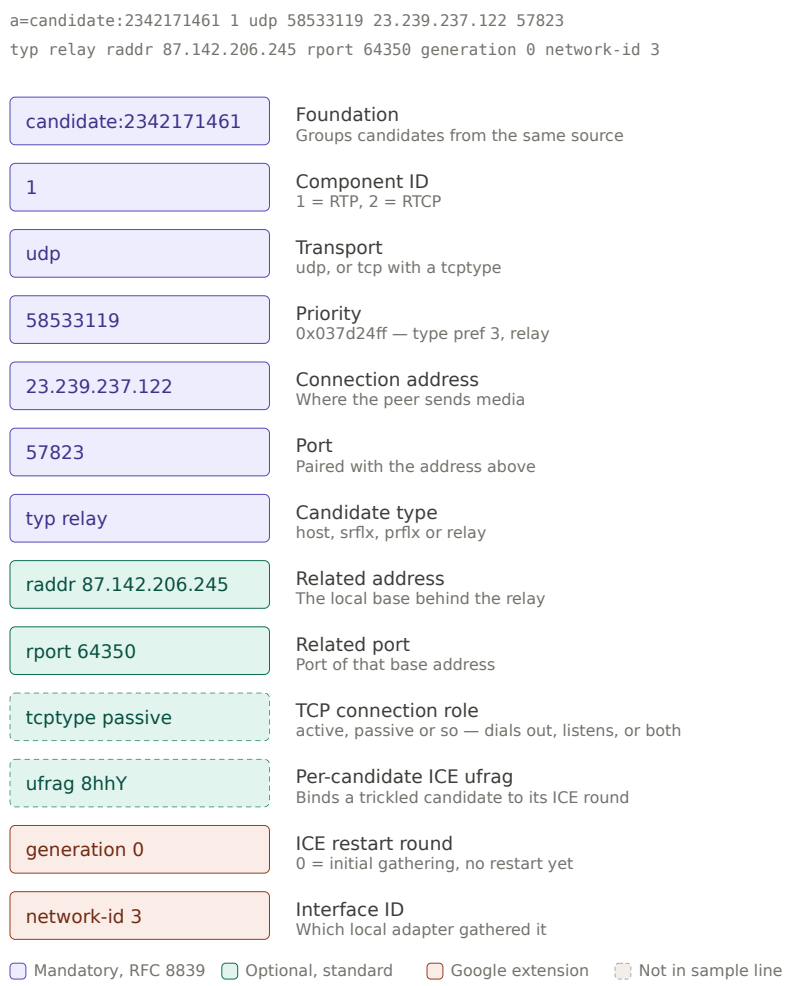
Glossary

Acronym	Full name	What it is
ICE	Interactive Connectivity Establishment	RFC 8445 - finding all possible ways for a peer connection to connect, weight them and choose the best
ICE candidate	ICE Candidate	one possible path endpoint - IP, port, protocol and type (host, srflx, prflx, relay) plus a priority
SDP	Session Description Protocol	RFC 8866 - Describes the parameters for connection establishment - Transport endpoint, type, codec, bitrate
STUN	Session Traversal Utilities for NAT	RFC 8489 - uses a specific STUN server which replies to specific packets with the IP/port it sees this packet coming from. UDP and TCP - the packet to the STUN server also creates a NAT binding, hence it’s not only the address, but also the port that is important. Used by the ICE protocol
TURN	Traversal Using Relays around NAT	RFC 8656 - relay server for media when no direct path works, reached over UDP, TCP or TLS. Used by the ICE protocol
NAT	Network Address Translation	RFC 4787 - rewrites private addresses/ports to public ones; its mapping behavior decides whether a direct ICE path works
DNS	Domain Name System	RFC 1034, RFC 1035, Port TCP/UDP 53
DoH	DNS over HTTPS	RFC 8484 - Tunneling DNS requests over HTTPS - more secure, harder to debug in pcap
TLS	Transport Layer Security	encryption for TCP connections current version 1.3 (RFC 8446) - version 1.0 & 1.1 are deprecated
DTLS	Datagram Transport Layer Security	TLS over UDP - exchanges the SRTP keys and carries the data channels, current version DTLS 1.3 (RFC 9147)
RTP	Real Time Transport Protocol	RFC 3550 - A Transport Protocol for Real-Time Applications - most widely used - typically over UDP
RTCP	Real Time Transport Control Protocol	RFC 3550 - Reception statistics and feedback for RTP media streams
SRTP, SRTCP	Secure RTP/RTCP	RFC 3711 - encrypted RTP/RTCP, keys negotiated via DTLS (DTLS-SRTP, RFC 5764)
SCTP	Stream Control Transmission Protocol	RFC 9260 - carries the WebRTC data channels, runs over DTLS (RFC 8261)
HTTP/2, H2	HTTP v 2	RFC 9113 - HTTP TCP-based (port 443), TLS encrypted
HTTP/3, H3	HTTP v 3	RFC 9114 - Latest major HTTP version, over QUIC on UDP, uses TLS 1.3

Acronym	Full name	What it is
QUIC	QUIC	RFC 9000 - A UDP-Based Multiplexed and Secure Transport
SSE	Security Service Edge	Cloud-based security framework
SASE	Secure Access Service Edge	Combines SSE with network routing capabilities

ICE candidates

Contained in the SDP and in the OnIceCandidate object in WebRTC Internals (chrome://webrtc-internals/).



Criteria in order of ranking

- **type** - how address was derived - *host, prflx, srflx, relay(UDP), relay(TCP), relay(TLS)*
- **network adapter** - Ethernet/wired, WiFi, VPN, Cell 5G/4G, Cell 2G/3G
- **address** - IPv6 loopback, IPv6 ULA, global IPv6, IPv4, IPv4 mapped IPv6, Teredo

> These criteria are layered: *type* is the highest ranking, *network adapter* only counts when there are two candidates of the same type, same for *address*.

Display & Capture Filters

Useful display filters

DeNoise

Remove stuff not needed for your analysis - in a capture from a clean VM or with proper capture filters, this will not be necessary
`!arp &&!ssdp &&!mdns&&!llmnr||!dhcp`

STUN & RTP

`stun` gives you the ICE setup
`rtp` the media streams - if the rtp_udp heuristic is enabled in `Analyze -> Enabled Protocols...`
`stun || rtp`

SDP

If you have the matching custom dissector installed you can just filter for `sdp`

Copy only one lua plugin at a time in Wiresharks personal LUA plugin directory, otherwise you may get double matches and increase scan time.

Media server

Take the media server's address from *this* capture - the selected pair in the ICE grid, or the candidates in the SDP answer - and filter on it, e.g. `ip.addr == 48.208.184.155`. Published address ranges drift and can miss the server entirely (see MS Teams).

Product specific

Each product section has its own filters for the parts that differ: where the signaling hides, TURN and gateway hostnames (SNI), and dissector quirks.

Capture filters

Use with care lest you remove relevant clues - a capture filter cannot be undone.

There are deliberately no per-product capture filters: address and port lists miss the fallbacks (TURN over TCP/TLS 443), the DNS lookups that show how a server was picked, and servers outside the published ranges. Ideally capture everything on a clean VM and narrow it down with display filters instead.

Obviously there are special cases where capture filters are required:

- Captures on live systems with lots of background traffic (you may consider not to dump TLS keys on such a system for privacy reasons)
- Hours long captures where you only want very specific data (consider to capture headers only, if that gives you the data you nee. Use capture size limit to create a sequence of capture files)

Remove irrelevant traffic

This one removes traffic you usually do not need (if you capture from a prepared clean VM, this is not necessary)
Mainly noisy broadcast and multicast as well as DHCP (67/68), NTP (123), NetBIOS (137/138), SSDP (1900), LLMNR (5355) and ARP.

```
(ip or ip6) and not broadcast and not multicast and not udp port 67 and not udp port 68 and not udp port 123 and not udp port 137 and not udp port 138 and not udp port 1900 and not udp port 5355 and not arp
```

My Capture setup

I wanted to capture on Windows and had a Windows host, hence it’s Hyper-V.

Linux VM on Linux host would have also worked, as all I did was in the browser.

Native apps typically require Windows or macOS.

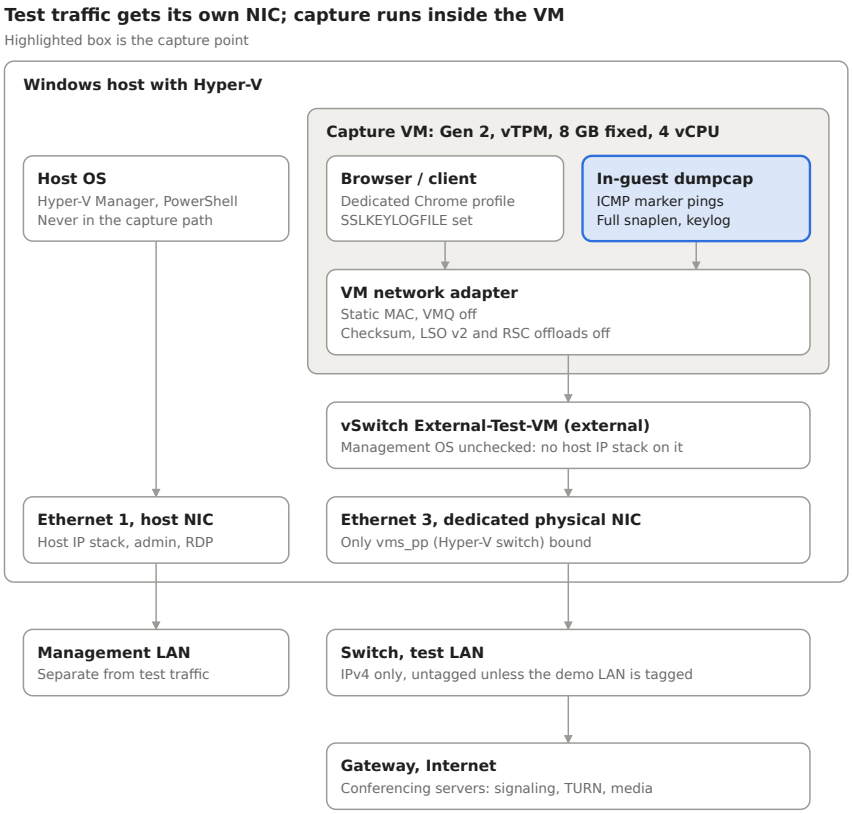
This is just an overview to give you an idea, ask your favorite AI for more guidance.

Setup at a glance

The capture VM owns a dedicated physical USB NIC through an external vSwitch the host cannot use, so only guest traffic reaches the Lab VLAN.

NIC is connected to a separate Lab VLAN to allow flexible firewall configuration between totally open to more restricted configs.

All captures run inside the VM, next to the browser. The captures described below were made without any network restrictions.



Most relevant configurations

Nine settings to keep the capture clean of unwanted background noise

Area	Setting	Why it matters
Host vSwitch	External vSwitch on the dedicated NIC, `-AllowManagementOS $false`; only `vms_pp` bound	Host traffic never enters the capture path
VM adapter	Static MAC, VMQ off	Stable guest identity across takes; no offload reordering
VM	Gen 2 + Secure Boot + vTPM, fixed 8 GB RAM, 4 vCPUs	Predictable performance for browser media
VM	Time Synchronization service off, guest uses real NTP	Capture timestamps line up with client logs and WebRTC Internals
Guest network	IPv6 off, Teredo/6to4/ISATAP off, NetBIOS, LLMNR and mDNS off, Public profile, metered	Calm ICE candidate list, no discovery chatter
Guest services	DiagTrack, DoSvc, SSDPSRV, FDResPub, upnphost, CDPSvc disabled	Removes the noisiest background traffic; recheck after reboot
Guest apps	Windows Update paused, OneDrive gone, local account, no BitLocker, dedicated Chrome profile	No sync, update or account traffic mid-take
Guest NIC offloads	Checksum, LSO v2 and RSC off	No false bad-checksum or giant-segment artifacts in the capture
Capture	`dumpcap`, filter `not broadcast and not multicast`, full snaplen, `SSLKEYLOGFILE` set, ICMP marker pings	Decryptable signaling, phase bookmarks

Capture automation scripts are here (Windows is the most complete).

(https://github.com/rohess/scripts/tree/main/ssldump)

MS Teams Cheat Sheet

Connect String

```
https://teams.live.com/light-meetings/launch?
p=ALos3Pb6N87m9Oq0l2&launchType=web&launchAgent=join_launcher_web&lightExperience=tru
e&anon=true&agent=web&coords=eyJtZ...e1ce0,
  "iceServers[{"urls":[
"turn:eu.relay.skype.com:3478?transport=udp",
"turn:eu.relay.skype.com:443?transport=tcp",
"turns:eu.relay.skype.com:443"
]}],"bundlePolicy":"max-bundle","alwaysNegotiateDataChannels":false},
```

Geo Distribution

Geo DNS and server side control - client sends its location.

Signaling

Different for each direction

- H2/JSON for Client -> Server
- WebSocket for Server -> Client

Log files

MS Teams has somewhat useful logs and can create a local log dump from browser via [Cmd|Ctrl]+[Option|Alt]+Shift+1

Ports & Addresses

Media (preferred)	Fallback / signaling	Notes
UDP 3478–3481	TCP 443, 80; UDP 443	Auth and other M365 sets are also needed (TCP 443)

IPv4	IPv6
52.112.0.0/14, 52.122.0.0/15	2603:1063::/38 (media). Signaling adds 2603:1027::/48, 2603:1037::/48, 2603:1047::/48, 2603:1057::/48, 2620:1ec:6::/48, 2620:1ec:40::/42

The full list of MS endpoints: (https://endpoints.office.com/endpoints/worldwide?clientrequestid=b10c5ed1-bad1-445f-b386-b919946339a7)

However if you configure packet filter according to the IPv4 list, you end up with TCP over TURN. In an unrestricted capture we see UDP 48.208.184.155 (Azure Sweden) for the media server - which is not on the list. There are some discussions in MS support, but nothing conclusive.

The address is in the overall Azure list - not very useful for your firewall config:

Tag	Prefix	Region
AzureCloud.swedencentral	48.208.184.0/22	Sweden Central
AzureCloud	48.192.0.0/11	(all regions)

Domains

- *.lync.com
- *.teams.cloud.microsoft
- *.teams.microsoft.com
- teams.cloud.microsoft
- teams.microsoft.com
- *.keydelivery.mediaservices.windows.net
- *.streaming.mediaservices.windows.net
- adl.windows.com
- join.secure.skypeassets.com
- mlccdnprod.azureedge.net
- *.skype.com

Display Filters

Signaling needs the TLS keys; `sdp` and `teams_trouter` need the Lua plugin for Teams.

What	Filter
Signaling entry (TLS handshake)	`tls.handshake.extensions_server_name == "api.flightproxy.skype.com"`
Trouter push channel (TLS handshake)	`tls.handshake.extensions_server_name contains "trouter"`
Trouter messages - answer and events	`teams_trouter`
SDP offer/answer	`sdp`
TURN relay lookup	`dns.qry.name contains "relay.skype.com"`
TURN over TLS	`tls.handshake.extensions_server_name contains "relay.skype.com"`
STUN 300 Try Alternate redirect	`stun.att.error.class == 3 && stun.att.error == 0`

ICE Grid

ICE Candidate pair: 80.136.59.151:3234 <=>48.208.184.155:3478

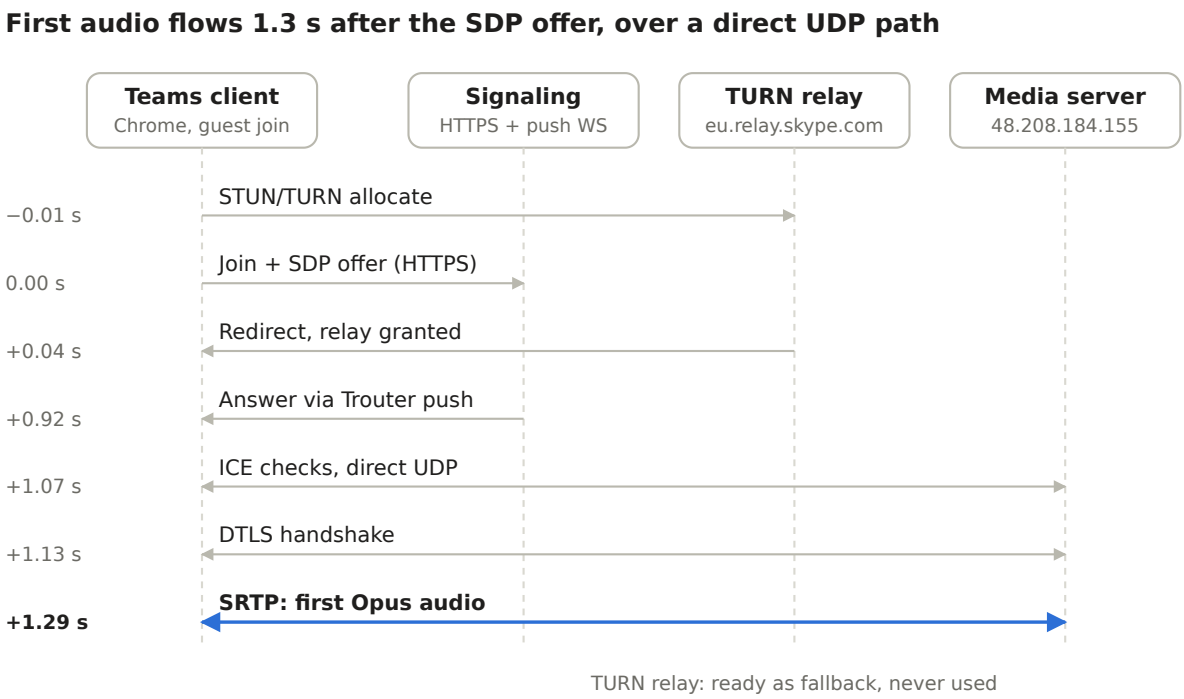
▼ ICE candidate grid

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type	(Pair) Priority	Bytes sent / received	STUN requests sent / responses received	STUN requests received / responses sent	RTT
CPJclY5sir_/pyeefvl	**succeeded**	**ethernet**		**udp**	**0x337ffffdcfe3e00**	**6333655 / 30938631**	**43 / 43**	**2 / 2**	**0.037s**
IJclY5sir	local-candidate	80.136.59.151	3234	prflx	0x6e7f1eff 110 | 32542 | 255				
l/pyeefvl	remote-candidate	48.208.184.155	3478	relay	0x337ffff				
CP4VHSaPVd_/pyeefvl	waiting	ethernet		udp	0x337ffff06fe4400	0 / 0	0 / 0	0 / 0	
I4VHSaPVd	local-candidate	20.202.140.198	52763	relay(udp) turn:eu.relay.skype.com:3478?transport=udp	0x37f21ff 3 | 32545 | 255				
l/pyeefvl	remote-candidate	48.208.184.155	3478	relay	0x337ffff				
CP8oHRegxk_4T62H3G5	waiting	ethernet		tcp active	0x113ffffb4fe3e00	0 / 0	0 / 0	0 / 0	
I8oHRegxk	local-candidate	192.168.102.78	9	host	0x5a7f1eff 90 | 32542 | 255				
I4T62H3G5	remote-candidate	48.208.184.155	3478	relay	0x113ffff				
CPAhX/4oQA_/pyeefvl	waiting	ethernet		udp	0x17f20ff06700000	0 / 0	0 / 0	0 / 0	
IAhX/4oQA	local-candidate	20.202.8.194	50159	relay(tcp) turn:eu.relay.skype.com:443?transport=tcp	0x17f20ff 1 | 32544 | 255				
l/pyeefvl	remote-candidate	48.208.184.155	3478	relay	0x337ffff				
CPdZuNXXgj_/pyeefvl	waiting	ethernet		udp	0x7f1fff06700000	0 / 0	0 / 0	0 / 0	
IdZuNXXgj	local-candidate	20.202.8.196	58381	relay(tls) turns:eu.relay.skype.com:443?transport=tcp	0x7f1fff 0 | 32543 | 255				
l/pyeefvl	remote-candidate	48.208.184.155	3478	relay	0x337ffff				

MS Teams Web Call Setup at a Glance

A Teams web client (Chrome) joining a meeting goes from sending its SDP offer to first audio in about 1.3 seconds. It connects straight to a Microsoft media server over UDP, and it never uses its TURN relay.

Setup sequence



TURN gathering starts just before the offer goes out. Most of the wait is the 0.9 s for the conference service to answer. After that, ICE, DTLS and first media take under 0.4 s.

Servers and selection

Every hop lands in a nearby Azure region. These were picked by DNS and by server-side redirects, not by the client.

Role	Server seen	How it was chosen
Signaling entry	api.flightproxy.skype.com	One global front door that forwards to regional services (Sweden Central, France Central, North Europe)
Push channel (Trouter)	Germany West Central	WebSocket opened at app start; the server delivers the answer and events over it

Role	Server seen	How it was chosen
TURN	eu.relay.skype.com	Azure Traffic Manager DNS (West Europe, France Central), then a STUN 300 Try Alternate redirect to another relay
Media server	48.208.184.155:3478	Chosen by the conferencing service; it appears only as the remote ICE candidate in the answer

The client also sends its country (clientLocation “DE”) with the join request.

ICE: trickle, lite and NAT

This call shows the optimal case: all relevant ports are open and the servers are close (short RTT), so the setup is minimal and trickle ICE and TURN have no effect.

- **Trickle ICE is announced but barely used.** The offer says a=ice-options:trickle, yet it carries only the private host candidate (192.168.102.78). No server-reflexive or relay candidates show up in the signaling.
- **TURN is gathered but stays idle.** The client gets UDP, TCP and TLS relays (all on port 443 or 3478) as a fallback for strict firewalls. In this call their candidate pairs never left “waiting”.
- **The media server is not ICE-lite here.** It sends its own connectivity checks back to the client, which a lite agent usually does not. It does advertise its public address as a relay-type candidate (although it’s not the classical TURN server).
- **The NAT gives each destination its own public port** (APDM - Address/port-dependent mapping). So the STUN-learned address is useless. The working path uses a peer-reflexive candidate, the address the media server saw during the checks.
- **Result:** a direct UDP path with 37 ms RTT. DTLS finishes and media starts about 0.4 s after the answer.

Codecs

Opus carries the audio and H.264 the video. VP8 and VP9 are not offered at all.

Media	Offered	Used in this call
Audio	Opus (with FEC and RED), G.722, G.711, comfort noise, DTMF	Opus, both directions
Video	H.264 (High, Main, Baseline; High 4:4:4 for screen share), AV1	H.264 High received, with RTX
Data	SCTP over DTLS (Microsoft’s own x-data m-line)	Yes, right after DTLS

Different from mainline WebRTC

- **No offer/answer over a WebSocket.** The offer goes out in an HTTPS (HTTP/2) join request. The answer comes back as a callback over a separate push WebSocket (Trouter).
- **SDP is wrapped in JSON** (content type application/sdp-ngc-1.0). Fields outside the SDP override it, for example turning five video slots to receive-only.
- **13 m-lines are prepared up front:** audio, camera, nine incoming video slots, screen share and data. Bundled on one port, so remote videos can be added without renegotiating.

Zoom

Cheat Sheet

Connect String

Data/Video

```
https://app.zoom.us/wc/82980644206/start?from=pwa&ref_from=launch,
{"iceCandidatePoolSize":1,"alwaysNegotiateDataChannels":false}
```

Audio

```
 https://app.zoom.us/wc/82980644206/join?
ref_from=launch&from=pwa&wpk=wcpk{0}%26%26%26%26wcpkd58f3b6227b86301dd7781e092c5a8cd&
_x_zm_rtaid=WqTuvvdvT9-
_5nLa8lmfmw.1790078138111.5444e48997a035baf6a8ecac7d9e54b3&_x_zm_rhtaid=999,
{"bundlePolicy":"max-bundle","alwaysNegotiateDataChannels":false}
```

Geo Distribution

Client pings list of provided hosts. Call from Germany goes to US server, maybe because of free tier. Other calls from Pro organizers show servers in Frankfurt.

Signaling

WebSocket

Log files

Zoom logs are allegedly encrypted - only support has access - intercepting client telemetry over the network might be useful.

Ports & Addresses

Media (preferred)	Fallback / signaling	Notes
UDP 3478, 3479, 8801–8810	TCP 443, 8801, 8802; TCP 80/443 and UDP 443 to *.zoom.us	

IPv4	IPv6
49 prefixes (full list below)	2407:30C0::/32, 2600:9000:2600::/48, 2620:123:2000::/40

Full list of IPs (https://assets.zoom.us/docs/ipranges/ZoomMeetings.txt)

Domains (TCP 443)

- *.zoom.us
- *.zoom.com
- cdn.cookielaw.org
- googletagmanager.com
- gstatic.com
- googleapis.com
- *.hcaptcha.com

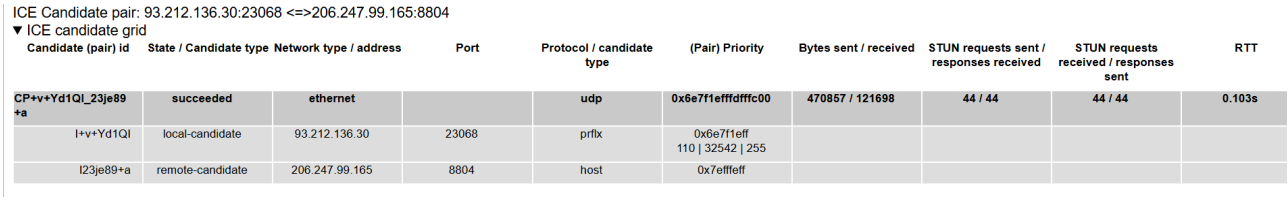
Zoom network firewall or proxy server settings (KB0060548)
(https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0060548)

Display Filters

The region probe needs the TLS keys; `sdp` and `zoom_ws` need the Lua plugin for Zoom.

What	Filter
Region probe	`http.request.uri contains "/wc/ping"`
Signaling gateway (TLS handshake)	`tls.handshake.extensions_server_name contains "rwg"`
Signaling messages	`zoom_ws`
SDP offer/answer	`sdp`
Media - audio and data channel	`udp.port in {8801..8810}`

ICE Grid



ICE Candidate pair: 93.212.136.30:23068 <=>206.247.99.165:8804
▼ ICE candidate grid

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type	(Pair) Priority	Bytes sent / received	STUN requests sent / responses received	STUN requests received / responses sent	RTT
CP+v+Yd1QI_23je89+a	succeeded	ethernet		udp	0x6e7f1efffdfffc00	470857 / 121698	44 / 44	44 / 44	0.103s
I+v+Yd1QI	local-candidate	93.212.136.30	23068	prflx	0x6e7f1eff 110 | 32542 | 255				
I23je89+a	remote-candidate	206.247.99.165	8804	host	0x7efffeff				

Zoom Web Call Setup at a Glance

A Zoom web client (Chrome) builds its media paths about 1 s after the first SDP offer. That is roughly 19 s before the user actually clicks “Join”. It connects straight to a Zoom media server in Ashburn (US East) over UDP and never uses its TURN relay. Audio is transported over a regular WebRTC audio channel, video via a WebRTC data channel.

Setup sequence



Zoom web join · times relative to the first SDP offer

Each offer/answer takes one round trip (about 100 ms). ICE and DTLS follow immediately. Both media paths are then kept alive while the user sits on the preview screen. After the join, first audio flows within 1.8 s.

Servers and selection

The whole call from Germany landed in one US datacenter (IAD). The client chose it from the two US sites it was offered to ping.

Role	Server seen	How it was chosen
Web app	app.zoom.us	Fixed entry point. Serves the join page and the app
Region probe	rwciad.iad.zoom.us, rwcsjc.sjc.zoom.us	The client pings both (`/wc/ping`). IAD won, although the client is in Germany: the meeting is hosted on Zoom's `us05` cluster, so only US sites are offered
Signaling	zoomiad20624799165rwg.iad.zoom.us	WebSocket gateway. Its hostname encodes its IP address
Media server	206.247.99.165, UDP 8801 and 8804	**The same IP as the signaling gateway.** It appears only as host candidates in the SDP answers
TURN	turniad01/02.iad.zoom.us (3478 UDP, 443 TLS)	Sent over the WebSocket, twice. Never contacted

ICE: trickle, lite and NAT

- **Trickle ICE is announced but not used.** The offers say `a=ice-options:trickle`, yet they carry no candidates, and none are sent later. The server learns the client's address from the client's own connectivity checks.
- **The media server is ICE-lite.** It lists its public address (UDP, plus TCP 8801 as a fallback) in the answer, and the browser does all the checking. Oddly, the audio server still sends checks back, while the data-channel server never does.
- **No STUN or TURN servers are contacted.** The client gathers only host candidates. Its public address appears as a peer-reflexive candidate, learned from the media server's reply to the first check.
- **Result:** a direct UDP path with 103 ms RTT (Germany to US East). ICE plus DTLS take about 0.3 s per connection.

Codecs

Audio uses Opus only. Video does not appear in the WebRTC SDP at all.

Media	Offered	Used in this call
Audio	Opus (with FEC and RED), G.722, G.711, comfort noise, DTMF	Opus only (the answer drops the rest). Mic at wideband 48 kbps, share audio at fullband 96 kbps, three stereo receive streams
Video	Not negotiated as RTP	None in the SDP. Video traffic runs over the data channel
Data	SCTP over DTLS, in its own connection	Carries the video, in Zoom's own format

Different from mainline WebRTC

- **Two peer connections instead of one bundle:** one for the data channel (video), one for audio. Each is negotiated over its own WebSocket.
- **SDP wrapped in Zoom's own framing:** binary records containing JSON, inside the WebSocket. The audio SDP is additionally gzip-compressed and base64-encoded, so searching for `v=0` finds nothing.
- **Media is pre-warmed:** both paths are fully connected while the user is still on the preview screen, so joining needs no new negotiation.
- **Fixed receive slots:** three receive-only audio streams are set up front, and the server fills them with the active speakers. Nothing is renegotiated when people talk.

Webex

Cheat Sheet

Connect String

Meeting URL (personal room, guest join from Chrome)

https://meet1754096804582-5056.webex.com/meet/pr27407383594 → redirects to `global-nebulabg.webex.com`, then the web app loads from `web.webex.com`

ICE server (seen in the ICE grid, handed out by Webex at join)

turns:xlm120.public.wlhrm-p-2.prod.infra.webex.com:443?transport=tcp

Geo Distribution

The client pings a list of media clusters it gets from Webex (Calliope): 16 nodes in 8 clusters around the world, on UDP 5004 and 9000. It does this twice, once on the pre-join screen and once at join. Call from Germany went to a media node in London, although Amsterdam answered fastest. The media node is assigned by Webex, not by the client.

Signaling

HTTPS REST (Locus) carrying SDP in a JSON envelope called ROAP. A WebSocket (Mercury) is used for event push, not for the SDP.

Log files

Webex desktop app has local logs.

Ports & Addresses

Media (preferred)	Fallback / signaling	Notes
SRTP/UDP 5004, 9000	SRTP/TCP 5004, then SRTP/TLS 443; HTTPS 443; DNS 53, NTP 123	Video Mesh Node uses UDP 50000–53000

IPv4	IPv6
18 prefixes - full list below	2402:2500::/34, 2607:fcf0::/34, 2a00:a640::/34

Network Requirements for Webex Services (WBX000028782) (https://help.webex.com/en-us/article/WBX000028782/Network-Requirements-for-Webex-Services)

Domains (TCP 443)

- *.webex.com
- *.cisco.com
- *.wbx2.com
- *.ciscospark.com
- *.webexapis.com
- *.webexcontent.com

Events

- *.livestream.webex.com
- *.vbrickrev.com

Livestreams

- *.livestreaming.webex.com

Display Filters

Signaling needs the TLS keys; `sdp` needs the Lua plugin for Webex.

What	Filter
Offer/answer request (Locus)	`http2.header.value contains "/locus/api/v1/loci"`
SDP offer/answer	`sdp`
Reachability probes	`stun.type == 0x0001 && !stun.att.username`
ICE checks on the media node	`stun.att.username contains ":"`
TLS to media nodes - probes and TURN	`tls.handshake.extensions_server_name contains "xlm"`
Media path	`(udp.port == 5004 || udp.port == 9000) && (dtls || rtcp || rtp)`

ICE Grid

ICE Candidate pair: 80.136.59.151:64079 <=>144.196.72.136:5004

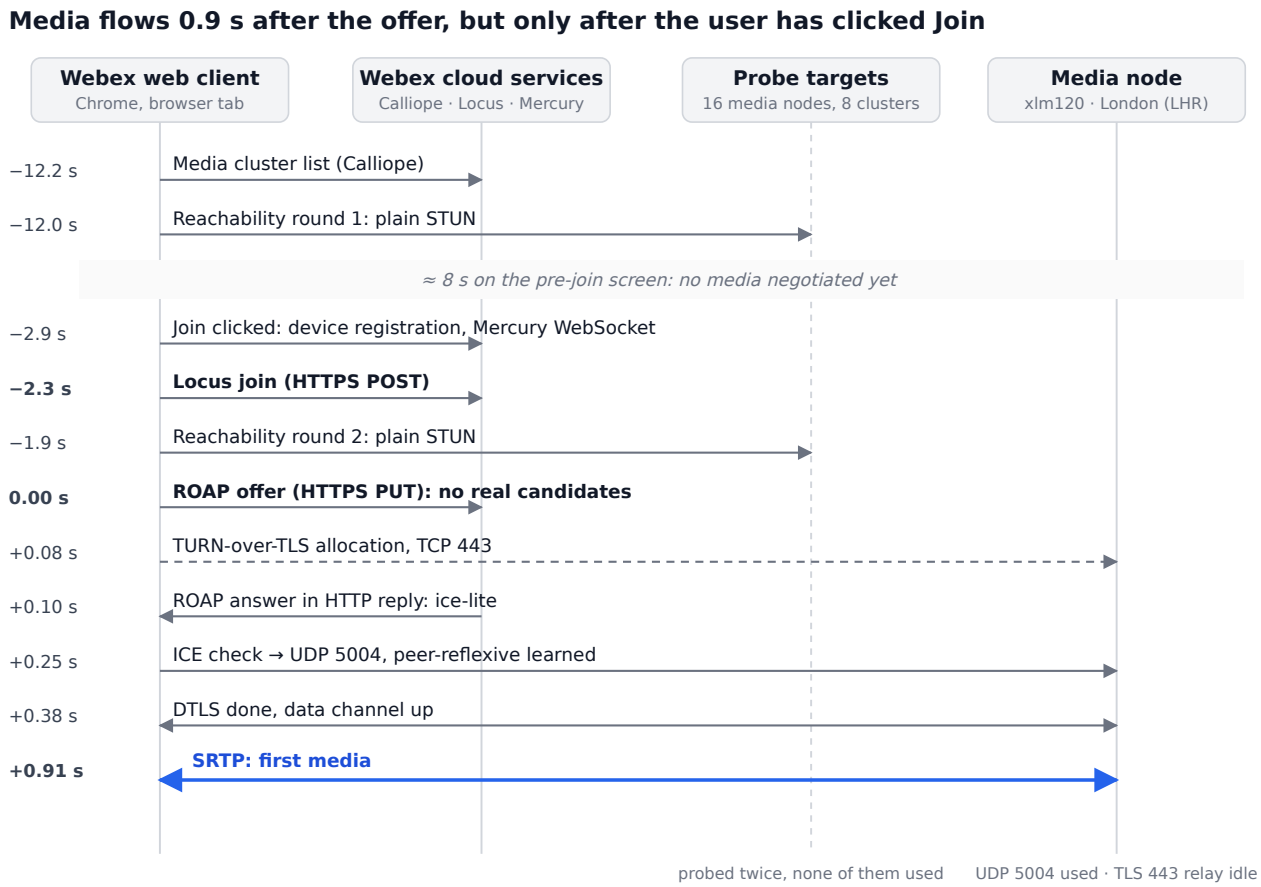
▼ ICE candidate grid

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type	(Pair) Priority	Bytes sent / received	STUN requests sent / responses received	STUN requests received / responses sent	RTT
CPAgjCyFVf_TKb4h1FW	**succeeded**	**ethernet**		**udp**	**0x6e7f1efffe000000**	**4300195 / 9684362**	**49 / 49**	**0 / 0**	**0.033s**
IAgjCyFVf	local-candidate	80.136.59.151	64079	prflx	0x6e7f1eff 110 | 32542 | 255				
ITKb4h1FW	remote-candidate	144.196.72.136	5004	host	0x7effffff				
CPiBBCxs0q_TKb4h1FW	waiting	ethernet		udp	0x7f20fffe000000	0 / 0	0 / 0	0 / 0	
liBBCxs0q	local-candidate	144.196.72.136	443	relay(tls) turns:xlm120.public.wlhrm-p-2.prod.infra.webex.com:443?transport=tcp	0x7f20ff 0 | 32544 | 255				
ITKb4h1FW	remote-candidate	144.196.72.136	5004	host	0x7effffff				
CPiBBCxs0q_oblKaaGV	waiting	ethernet		udp	0x7f20fffdfffa00	0 / 0	0 / 0	0 / 0	
liBBCxs0q	local-candidate	144.196.72.136	443	relay(tls) turns:xlm120.public.wlhrm-p-2.prod.infra.webex.com:443?transport=tcp	0x7f20ff 0 | 32544 | 255				
IoblKaaGV	remote-candidate	144.196.72.136	9000	host	0x7efffcff				
CPnJY1F8+8_0wgnD0ON	waiting	ethernet		tcp active	0x5a7f1effe9fff400	0 / 0	0 / 0	0 / 0	
InJY1F8+8	local-candidate	192.168.102.78	9	host	0x5a7f1eff 90 | 32542 | 255				
l0wgnD0ON	remote-candidate	144.196.72.136	5004	host	0x74fff9ff				
CPoQhKOe1O_oblKaaGV	succeeded	ethernet		udp	0x6e7f1efffdfff800	0 / 0	1 / 1	0 / 0	0.034s
loQhKOe1O	local-candidate	80.136.59.151	17664	prflx	0x6e7f1eff 110 | 32542 | 255				
IoblKaaGV	remote-candidate	144.196.72.136	9000	host	0x7efffcff				

Webex Web Call Setup at a Glance

A Webex web client (Chrome) only negotiates media after the user clicks “Join”. First media flows 0.9 s after the SDP offer and about 3.2 s after the join request. Before that, the client measures its distance to Webex media clusters twice. The call goes straight to a Webex media node in London over UDP 5004. A TURN relay on TLS 443 is set up as a backup but never used.

Setup sequence



Webex web join · times relative to the SDP offer

The offer goes out as an HTTPS request, and the answer comes back in the same request’s reply after about 100 ms. ICE and DTLS follow immediately and take about 0.3 s together.

Servers and selection

The client measured eight clusters around the world, but the media node it got was not one of the nodes it measured. This suggests that only one host per cluster is measured, and load balancing then picks the server within the cluster. Webex also did not pick the cluster with the shortest RTT but the second best, which suggests load balancing between clusters as well.

Role	Server seen	How it was chosen
Web app	web.webex.com (CloudFront)	Fixed entry point. Serves the app
Cluster list	calliope-anycast.prod.infra.webex.com	Anycast. Returns the media clusters to probe
Region probe	16 nodes `xlm*.public.<cluster>…`, UDP 5004 and 9000	Plain STUN. Cluster names look like airport codes: Amsterdam 21 ms, London 27 ms, New York ~110 ms, Dallas, Mumbai, San Jose ~170 ms
Signaling	Locus via `locus-k.wbx2.com` → AWS Frankfurt (170.72.238.49)	Nearest regional entry point. Carries offer and answer
Media server	xlm120.public.wlhrm-p-2 (London), 144.196.72.136, UDP 5004	**Assigned by Webex.** Not in the probe set. The probe results are most likely reported to Webex, which picks the cluster
TURN	The same media node, TLS 443	Credentials handed out before the offer. Allocated, never used

ICE: trickle, lite and NAT

- **Trickle ICE is announced but not used.** The offer says `a=ice-options:trickle`, yet it carries only placeholder candidates (`0.0.0.0` port 9), and no real ones are sent later. The server learns the client's address from the client's own connectivity checks.
- **The media server is ICE-lite.** Its answer lists its own addresses: UDP 5004 and 9000, TCP 5004, IPv4, IPv6 and a hostname. The browser does all the checking.
- **The client's public address is peer-reflexive.** It appears from the media node's reply to the first check, which is the only selected pair in the ICE grid.
- **TURN over TLS 443 is prepared in parallel** while the offer is in flight. It finishes after the direct UDP path already works and stays idle.
- **Result:** a direct UDP path with 33 ms RTT (Germany to London). ICE plus DTLS take about 0.3 s.

Codecs

H.264 and Opus only. The browser's default offer would also contain VP8, VP9, AV1 and more, so Webex trims the codec list before sending.

Media	Offered	Used in this call
Audio	Opus with in-band FEC	Opus stereo with DTX. FEC on for the main audio, off for the screen-share audio
Video	H.264 only (Baseline, Constrained Baseline, Main), each with retransmission	H.264. Sent as 3 simulcast layers, up to ~180p, ~576p and 1080p. The server accepts up to 4K
Screen share	A second audio + video pair marked `content:slides`	H.264 and Opus, set up from the start
Data	SCTP over DTLS	Opens right after DTLS, before the first media

Different from mainline WebRTC

- **SDP over REST, not WebSocket:** the offer is an HTTPS request, and the answer comes back in its reply. The SDP sits inside a JSON string inside Webex's own JSON envelope (ROAP), so it has to be decoded twice.
- **The answer bends the standard:** it uses the plain `RTP/AVP` profile in its answer to a secure offer, and lists a non-standard `xTLS` candidate type on port 443 that the browser ignores.
- **Screen share is negotiated up front:** one bundle with main audio/video, screen-share audio/video and a data channel. Sharing later needs no new media lines.
- **Webex picks the server:** the client measures and Webex decides. The media node is fixed at join, which is why TURN credentials for it are ready before the offer.

Google Meet

Cheat Sheet

Connect String

Meeting URL

```
https://meet.google.com/rmu-ivbi-ban?pli=1&ijlm=1791466252329&hs=187&adhoc=1,
{"bundlePolicy":"max-bundle","alwaysNegotiateDataChannels":false}
```

Offer/answer RPC (HTTP/2, protobuf, no text SDP)

POST https://meet.google.com/$rpc/google.rtc.meetings.v1.MediaSessionService/CreateMediaSession

Geo Distribution

No client-side region probe. The server picks the media server and returns it as host candidates in the answer. Call from Germany (Workspace account) landed on 74.125.250.248, RTT 15–30 ms, so most likely a European site.

Signaling

HTTP/2 RPC to meet.google.com (gRPC-web style), protobuf payloads. Request gzip, response gzip + base64.

Log files

Meet - allegedly you can find something under chrome://webrtc-logs/ - I was not able to trigger this.

Ports & Addresses

Media (preferred)	Fallback / signaling	Notes
UDP 3478, 19302–19309	TCP 19305; TCP/UDP 443 (TURN over TLS via SNI)	Ports can be opened without an IP restriction

IPv4	IPv6
Workspace: 74.125.250.0/24, 74.125.247.128/32 (SNI workspace.turns.goog). Consumer: 142.250.82.0/24 (SNI meet.turns.goog)	Workspace: 2001:4860:4864:5::/64, 2001:4860:4864:4:8000::/128. Consumer: 2001:4860:4864:6::/64

Domains (TCP 443)

Domains for static resources

- clients2.google.com
- clients4.google.com
- clients6.google.com
- www.gstatic.com
- fonts.gstatic.com
- lh3.googleusercontent.com
- *.clients6.google.com

Domains for API endpoint connectivity

- accounts.google.com
- apis.google.com
- meetings.googleapis.com
- hangouts.googleapis.com
- meet.google.com
- apps.google.com
- chat.google.com
- docs.google.com

Domains for live streaming

- meet.google.com
- stream.meet.google.com

Domains for user feedback & event log uploads

- https://www.google.com/tools/feedback
- https://feedback.googleusercontent.com/resources/
- https://play.google.com/log

Full docs (https://knowledge.workspace.google.com/admin/meet/prepare-your-network-for-meet-meetings-and-live-streams?rd=1)

Display Filters

The RPCs need the TLS keys; `sdp` needs the Lua plugin for Meet.

What	Filter
Meeting lookup, before the offer	`http2.header.value contains "ResolveMeetingSpace"`
Offer/answer RPC	`http2.header.value contains "CreateMediaSession"`
SDP offer/answer	`sdp`

ICE Grid

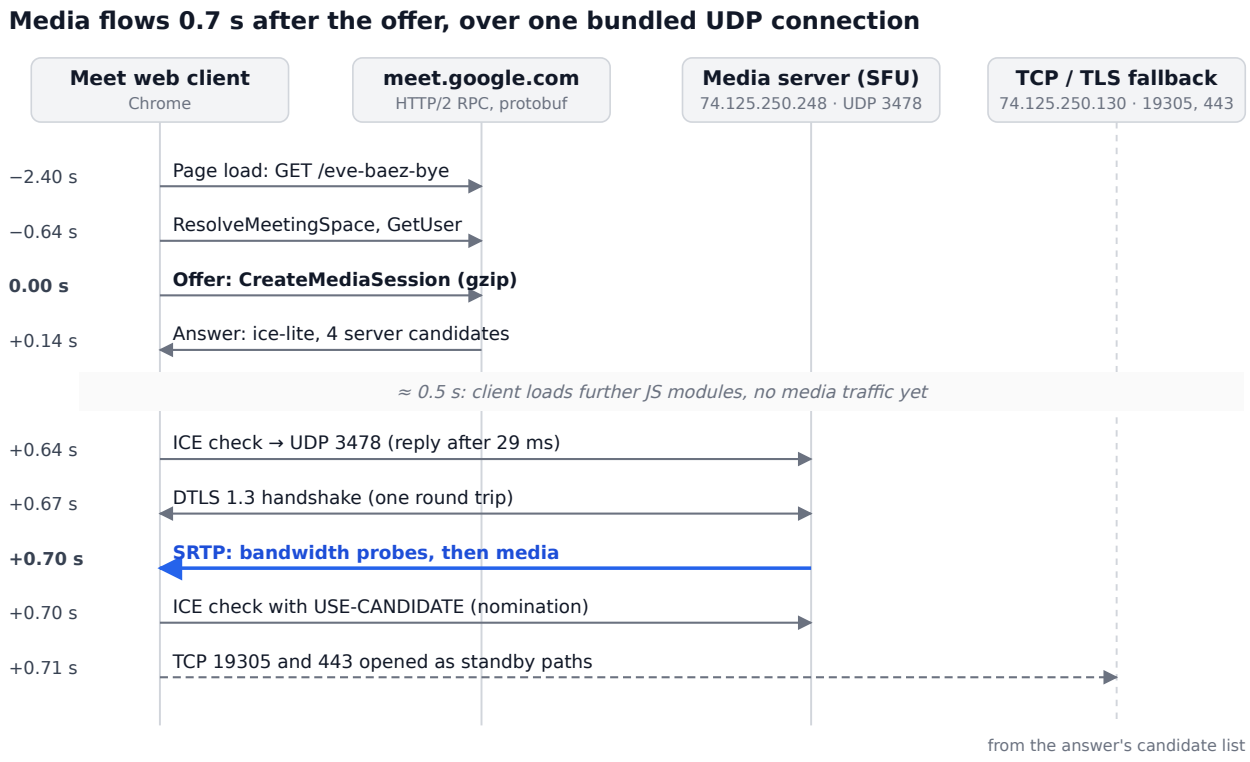
▼ ICE candidate grid

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type	(Pair) Priority	Bytes sent / received	STUN requests sent / responses received	STUN requests received / responses sent	RTT
CPTJr7pwOm_1YwQNOV6	**succeeded**	**ethernet**		**udp**	**0x6e7f1efffe000000**	**2298233 / 2667206**	**28 / 28**	**0 / 0**	**0.015s**
ITJr7pwOm	local-candidate	80.136.59.151	54705	prflx	0x6e7f1eff 110 | 32542 | 255				
I1YwQNOV6	remote-candidate	74.125.250.248	3478	host	0x7f000004				
CPZnKd6+Fb_OKe0PyH4	waiting	ethernet		tcp active	0x5a7f1efffe000000	0 / 0	0 / 0	0 / 0	
IZnKd6+Fb	local-candidate	192.168.102.77	9	host	0x5a7f1eff 90 | 32542 | 255				
IOKe0PyH4	remote-candidate	74.125.250.130	19305	host	0x7effffffe				
CPZnKd6+Fb_vcN7pBOz	waiting	ethernet		tcp active	0x5a7f1efffe000000	0 / 0	0 / 0	0 / 0	
IZnKd6+Fb	local-candidate	192.168.102.77	9	host	0x5a7f1eff 90 | 32542 | 255				
IvcN7pBOz	remote-candidate	74.125.250.130	443	host	0x7effffffd				
CPbA/jVFWR_FhjsvKOj	in-progress	ethernet		udp	0x7e7f1efffe000000	0 / 0	24 / 0	0 / 0	
IbA/jVFWR	local-candidate	192.168.102.77	54023	host	0x7e7f1eff 126 | 32542 | 255				
IFhjsvKOj	remote-candidate	74.125.250.248	19305	host	0x7effffff				

Google Meet Web Call Setup at a Glance

A Google Meet web client (Chrome) negotiates its media path with a single RPC that carries the offer as protobuf, not as SDP. One round trip later it has the answer. About 0.7 s after the offer, media flows directly to a Google media server over UDP. No STUN or TURN server is contacted. Audio, video and 14 data channels share one bundled connection.

Setup sequence



Google Meet web join · times relative to the offer

The offer/answer takes 140 ms. The client then spends about 0.5 s loading further app modules before the first ICE check. ICE and DTLS together take about 60 ms; the first media packets follow immediately.

Servers and selection

Role	Server seen	How it was chosen
Web app and signaling	meet.google.com	Fixed entry point. Serves the page and all RPCs over one HTTP/2 connection

Role	Server seen	How it was chosen
Meeting lookup	ResolveMeetingSpace RPC	Turns the meeting code into a meeting space before the offer
Media server	74.125.250.248, UDP 3478	**Chosen by Google, not the client.** Returned as the top candidate in the answer. Workspace range
Fallback paths	74.125.250.130, TCP 19305 and TLS 443	Also in the answer. The client opens both right away as standby, although UDP already works
STUN / TURN	none	Not contacted in this call

ICE: trickle, lite and NAT

- **No trickle ICE in practice.** The offer carries no client candidates, and none are sent later. The server learns the client's public address from the client's first connectivity check (a peer-reflexive candidate).
- **The media server is ICE-lite.** It lists four host candidates (UDP 3478, UDP 19305, TCP 19305, TLS 443) and never sends checks itself. The browser does all the checking.
- **The session ID is the ICE username.** The server's ICE ufrag equals the media session ID returned in the answer, so a STUN packet can be tied back to its signaling call.
- **Result:** a direct UDP path, about 15–30 ms RTT. DTLS 1.3 completes in one round trip.

Codecs

Media	Offered	Accepted in the answer
Audio	Opus (with in-band FEC), RED	Both
Video	VP8, VP9 (profiles 0 and 2), AV1, each with RTX	All of them
Data	SCTP over DTLS, 14 named channels (reactions, captions, chat, …)	All, with stream IDs fixed by the server

The answer does not narrow the codec list. Which codec is actually sent is decided later, and is hidden by SRTP encryption.

Different from mainline WebRTC

- **No SDP on the wire:** offer and answer are protobuf messages in an HTTP/2 RPC. The browser's SDP is converted in JavaScript. The response is additionally base64-encoded.
- **One bundled connection:** audio, video and all data channels share a single UDP 5-tuple. The data channels are pre-negotiated by the server, not opened in-band.

- **Fallbacks prepared in advance:** TCP and TLS paths are opened as soon as UDP succeeds, so a switch needs no new negotiation.
- **Probing before media:** the server starts with RTX padding packets to estimate bandwidth before sending real audio and video.
- **Fixed receive slots (likely):** the answer seems to list three audio SSRCs, which matches Meet forwarding the loudest few speakers as separate streams.

GoToMeeting

Cheat Sheet

Connect String

Audio (WebSocket signaling, one URL per audio connection)

```
wss://audio-ds-live-default-8684846bd6-fkmp8.k8s.goto-rtc.com/rtcgw/94cb965c-ecc9-
4cca-bf57-cf31356ff943,
{"iceServers":[
{"urls":["stun:turn.goto-rtc.com:3478"]},
{"urls":["turn:turn.goto-rtc.com:3478?transport=udp",
         "turn:turn.goto-rtc.com:1853?transport=udp",
         "turn:turn.goto-rtc.com:8200?transport=udp",
         "turn:turn.goto-rtc.com:3478?transport=tcp",
         "turn:turn.goto-rtc.com:443?transport=tcp",
         "turns:turn.goto-rtc.com:443",
         "turns:turn.goto-rtc.com:5349"]}]},
"disableDirectTcpConnections":true, "numberOfDownstreams":4
```

Geo Distribution

No ping-based probing. The audio entry point `audio-client-signaling-live.goto-rtc.com` resolves (DNS on AWS Route 53) to a Frankfurt cluster, which redirects the client to one specific audio pod. Audio signaling, media and TURN were all in Frankfurt (13 ms RTT from Dresden); only the meeting's application server was in the US (109 ms).

Signaling

WebSocket, carrying REST-style JSON requests (`PUT …/offer`, `PUT …/candidate`, long-poll `GET …/events`)

Log files

GoTo desktop app has comprehensive local logs - right-click on tray icon - from browser you can send a GoTo Issue Report which includes the log, but that's not stored locally.

Ports & Addresses

Media (preferred)	Fallback / signaling	Notes
UDP 1853, 8200	TCP 8200, 443, 80	TURN also offered on UDP/TCP 3478, TLS 443 and 5349

IPv4	IPv6
68.64.0.0/19 173.199.0.0/18 78.108.124.0/23 202.173.24.0/21 23.239.224.0/19	Not offered on server side

Domains (TCP 443)

General

- *.goto.com
- *.goto-rtc.com
- *.jive.com
- *.getgo.com
- *.gotomeeting.com
- *.logmeininc.com
- *.expertcity.com

Training

- *.gototraining.com
- *.firebase.app
- *.youtube.com
- *.googlevideo.com

Webinar

- *.gotowebinar.com
- *.recordingassets.logmeininc.com

Minimal configuration (https://support.goto.com/webinar/help/minimal-firewall-settings-for-the-goto-app)

Display Filters

Set `Preferences -> Protocols -> WebSocket -> Dissect websocket text as -> JSON` first, and have the TLS keys. Wireshark shows STUN on UDP 8200 as “ADP”: use `Analyze -> Decode As... -> UDP port 8200 -> STUN`. `sdp` needs the generic JSON Lua plugin.

What	Filter
Signaling pod (TLS handshake)	`tls.handshake.extensions_server_name contains "audio-ds-live"`
Offer and answer	`json.value.string == "offer" || json.value.string == "answer"`
Trickled candidates	`json.value.string contains "candidate:"`
End of candidates	`json.value.string == "endOfCandidates"`

What	Filter
SDP offer/answer	`sdp`

ICE Grid

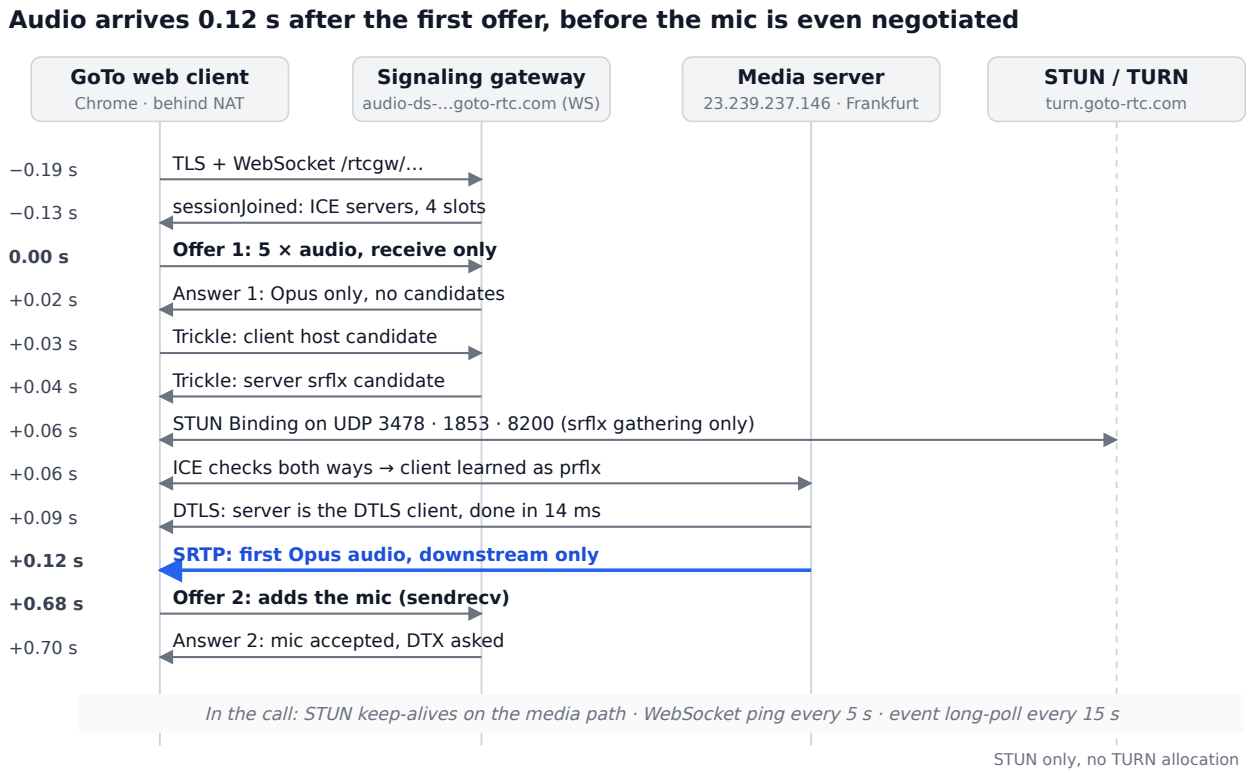
ICE Candidate pair: 93.212.136.30:25540 <=>23.239.237.146:45914
▼ ICE candidate grid

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type	(Pair) Priority	Bytes sent / received	STUN requests sent / responses received	STUN requests received / responses sent	RTT	Last data sent / received	Last update
CPMCC/xdtG_hK+xKYCE	**succeeded**	**ethernet**		**udp**	**0x647f3effdcfe3c00**	**151705 / 260307**	**22 / 22**	**23 / 23**	**0.013s**	**3:05:58 PM / 3:05:58 PM**	**3:05:58 PM**
IMCC/xdtG	local-candidate	93.212.136.30	25540	prflx	0x6e7f1eff 110 | 32542 | 255						
IhK+xKYCE	remote-candidate	23.239.237.146	45914	srflx	0x647f3eff						

GoToMeeting Web Call Setup at a Glance

A GoTo web client (Chrome) joins the audio **listening first**. The first offer is receive-only, and Opus audio from the server arrives 0.12 s later. A second offer, 0.7 s after the first, adds the microphone. Signaling, media and TURN all sit in GoTo's Frankfurt site, and the media flows directly over UDP without a TURN relay.

Setup sequence



GoTo web join, audio · times relative to the first SDP offer

Each offer/answer takes about 20 ms (one round trip plus processing). Candidates follow as separate trickle messages. ICE checks start as soon as the server's candidate arrives, and first audio follows 85 ms later. During the call the WebSocket carries keep-alives and event signaling: a ping every 5 s and an event long-poll every 15 s.

Servers and selection

All audio components were picked by DNS and an HTTP redirect, not by probing.

Role	Server seen	How it was chosen
Web app	app.goto.com	Fixed entry point. Origin of the WebSocket

Role	Server seen	How it was chosen
Application server	application-server-live-global.k8s.goto-rtc.com	Meeting control. A global service, here in the US (109 ms RTT)
Audio entry point	audio-client-signaling-live.goto-rtc.com → audio-ds-live.k8s.goto-rtc.com	DNS (AWS Route 53) returns a Frankfurt cluster. `POST /api/4/audio` answers with **307 Temporary Redirect** to one specific pod
Signaling	audio-ds-live-default-…-fkmp8.k8s.goto-rtc.com	The pod from the redirect (a Kubernetes pod name). `POST` → 201 Created, then the WebSocket
Media server	23.239.237.146, UDP 45914	**The same IP as the signaling pod.** Full ICE agent behind NAT (private 10.240.x address)
STUN / TURN	turn.goto-rtc.com → 5 Frankfurt IPs	Sent once in `sessionJoined`, with fixed credentials. Only STUN Binding requests, no TURN allocation as direct connect is faster

ICE: trickle, lite and NAT

- **Trickle ICE is used.** The first offer and answer carry no candidates. Both sides send them afterwards as separate messages, and the server closes with `endOfCandidates`.
- **The media server is *not* ICE-lite.** It runs full ICE: it sends its own connectivity checks to the client, announces a server-reflexive address (NAT from a private 10.240.x address) and takes the DTLS client role (`a=setup:active`).
- **The client only announces its host address.** It does ask GoTo's STUN server for its public address (on UDP 3478, 1853 and 8200), but never sends the result, as the direct setup via prflx is faster. The media server learns the client's public address from the client's own checks, so it shows up as **peer-reflexive** in the ICE grid.
- **No TURN, no TCP.** The direct UDP path worked, so no relay was allocated. Direct ICE-TCP is disabled in the config, so a TCP fallback would have to go through TURN (TCP 443 or TLS 443/5349).
- **Result:** a direct UDP path with 13 ms RTT. From the first server ICE candidate to the first audio downstream packet it takes 85 ms.

Codecs

Audio only. Video runs in a separate connection to a video gateway and is not covered here.

Media	Offered	Used in this call
Audio	Opus (stereo, FEC, RED), G.722, G.711, comfort noise, DTMF	Opus only (the answer drops the rest, including RED). FEC on, 20 ms packets. The server asks the client to use DTX for the mic. Downstream audio flows from 0.12 s; no upstream audio yet in the captured first second

Different from mainline WebRTC

- **REST over WebSocket:** every message carries a URL and HTTP method (`PUT …/offer`, `PUT …/candidate`). Answers and server candidates arrive as events on a long-polled `GET …/events`, all inside one WebSocket.
- **Listen first, talk second:** the first offer only receives audio, and audio is already flowing before the microphone is added by a renegotiation 0.7 s later.
- **Fixed receive slots:** the server announces four downstreams up front. The client offers five receive-only audio lines, the server fills four (one is inactive), plus one line for the mic. Nothing is renegotiated when people talk.
- **Server selection by redirect:** a regional DNS name plus an HTTP 307 to one pod, instead of the client probing several sites.

References

Class related:

- Handouts, Appendix, Sample Captures, Wireshark Profile, LUA plugins (https://github.com/rohess/sfeu26)
- As a single ZIP file: (https://bit.ly/4mOt7oR)

Articles, RFCs

RFC 4787: BCP 127: Network Address Translation (NAT) Behavioral Requirements for Unicast UDP (https://www.rfc-editor.org/info/rfc4787/)

Real-Time Compromise: Investigating RTC Security in Consumer IoT (https://www.usenix.org/system/files/woot26-goeman.pdf), Victor Goeman, Tom Cordemans, Christoph Sanders, Jorn Lapon, and Vincent Naessens, DistriNet, KU Leuven; 2026

SharkFest talks

Analyzing WebRTC connectivity on the packet level - a field report (https://github.com/rohess/sfeu23) - Robert Hess

Making WebRTC work in times of locked down networks (https://github.com/rohess/sfeu25) - Robert Hess

Tooling

How to enable GeoIP in Wireshark (https://wiki.wireshark.org/HowToUseGeoIP)

WebRTC-internals / rtcstats

Analyzing webrtc-internals Tsahi Levent-Levi 2026 (https://bloggeek.me/webrtcglossary/webrtc-internals/)

rtcstats github repo (https://github.com/rtcstats/rtcstats) - you can selfhost the basic version, no data leaves our systems.

TURN server security

Sandro Gauci - Enable Security:

- TURN Security Threats: A Hacker's View (https://www.enablesecurity.com/blog/turn-server-security-threats/)
- TURN Server Security Best Practices (https://www.enablesecurity.com/blog/turn-security-best-practices/)
- Securing coturn: Configuration Guide (https://www.enablesecurity.com/blog/coturn-security-configuration-guide/)

People

Relevant people and their publications (just my limited view on these things):

Tsahi Levent-Levi - bloggeek.me (https://bloggeek.me/) - weekly posts about WebRTC, focussed on SDKs, VpaaS

Philipp Hancke webrtchacks.com (https://webrtchacks.com/) - in depth analysis of Chrome and WebRTC workings and new features

Sandro Gauci enablesecurity.com (https://www.enablesecurity.com/)

Ckristian Zoli (Vonage) (https://www.linkedin.com/in/ckristianzoli/) - recently rather active on LinkedIn